CSCD 327 Lab 9

Dustin Randall

March 5, 2026

1 Find customer's total balance

```sql
DELIMITER $
CREATE FUNCTION sumBal(custId INT)
RETURNS DECIMAL(12, 2)
DETERMINISTIC
BEGIN
  DECLARE r DECIMAL(12, 2);
  SELECT SUM(balance) INTO r
  FROM accounts
  WHERE customerID = custId;
  RETURN r;
END $
DELIMITER ;
```

```sql
SELECT customerID, sumBal(customerID) AS totalBalance
FROM customers
WHERE customerID BETWEEN 1 AND 10;
```

*	customerID	totalBalance
1	1	1700
2	2	2400
3	3	1500
4	4	3300
5	5	2100
6	6	900
7	7	5000
8	8	1900
9	9	3400
10	10	2100

Figure 1: Customer Balances

2 Find customer's highest balance

```
DELIMITER $
CREATE FUNCTION maxBal(custId INT)
RETURNS VARCHAR(50)
DETERMINISTIC
BEGIN
  DECLARE sBal DECIMAL(12, 2);
  DECLARE cBal DECIMAL(12, 2);
  DECLARE r VARCHAR(50);

  SELECT balance INTO sBal
  FROM accounts
  WHERE customerID = custId AND accountType='Savings';
  SELECT balance INTO cBal
  FROM accounts
  WHERE customerID = custId AND accountType='Checking';

  IF sBal IS NULL AND cBal IS NULL THEN SET r = 'No Account Found';
  ELSEIF sBal IS NULL OR cBal IS NULL THEN SET r = 'One Account Found';
  ELSEIF sBal = cBal THEN SET r = 'Balances Are Equal';
  ELSEIF sBal < cBal THEN SET r = 'Checking Is Greater';
  ELSE SET r = 'Savings Is Greater';
  END IF;

  RETURN r;
END $
DELIMITER ;
```

```sql
SELECT customerID, maxBal(customerID) AS msg
FROM customers
WHERE customerID IN(1, 5, 8, 14);
```

*	customerID	msg
1	1	Savings Is Greater
2	5	One Account Found
3	8	Checking Is Greater
4	14	Balances Are Equal

Figure 2: Customer Highest Balances

3 Find transaction sums

```sql
DELIMITER $
CREATE PROCEDURE sumTrxs(custId INT)
DETERMINISTIC
BEGIN
  SELECT TransactionType, SUM(amount) AS total
  FROM transactions NATURAL JOIN accounts
  WHERE customerID = custId
  GROUP BY transactionType;
END $
DELIMITER ;
```

```sql
CALL sumTrxs(1);
CALL sumTrxs(5);
CALL sumTrxs(10);
```

*	TransactionType	total
1	Deposit	500.00
2	Withdrawal	200.00

Figure 3: Customer 1

*	TransactionType	total
1	Deposit	1000.00

Figure 4: Customer 5

*	TransactionType	total
1	Withdrawal	150.00

Figure 5: Customer 10

4 Make Deposit

```sql
DELIMITER $
CREATE TRIGGER applyTrx
AFTER INSERT ON transactions
FOR EACH ROW
BEGIN
  UPDATE accounts
  SET balance = CASE
    WHEN NEW.transactionType = 'Deposit' THEN balance + NEW.amount
    ELSE balance - NEW.amount
    END
  WHERE accountID = new.accountID;
END $
DELIMITER ;
```

```sql
DELIMITER $
CREATE PROCEDURE deposit(acctId INT, amt DECIMAL(12, 2))
DETERMINISTIC
BEGIN
  INSERT INTO transactions(accountID, amount, transactionType,
     transactionTimestamp)
  VALUES (acctId, amt, 'Deposit', NOW());
END $
DELIMITER ;
```

```sql
SELECT balance FROM accounts WHERE accountID = 1005;
```

*	balance
1	3000.00

Figure 6: Initial Balance

```sql
SELECT * FROM transactions WHERE accountID = 1005;
```

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10008	1005	400.00	Deposit	2025-10-06 13:22:00
2	10009	1005	200.00	Withdrawal	2025-11-21 14:00:00

Figure 7: Initial Transactions

```sql
CALL deposit(1005, 100.25);
```

*	balance
1	3100.25

Figure 8: Balance After Deposit

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10008	1005	400.00	Deposit	2025-10-06 13:22:00
2	10009	1005	200.00	Withdrawal	2025-11-21 14:00:00
3	10021	1005	100.25	Deposit	2026-03-05 22:20:27

Figure 9: Transactions After Deposit

5 Make Deposit

```
DELIMITER $
CREATE PROCEDURE withdrawl(acctId INT, amt DECIMAL(12, 2), OUT msg
   VARCHAR(20))
DETERMINISTIC
BEGIN
  DECLARE bal DECIMAL(12, 2);
  SELECT balance INTO bal
  FROM accounts WHERE accountId = acctId;

  IF amt < bal THEN SET msg = 'Success';
  ELSE SET msg = 'Insufficient Funds';
  END IF;

  IF amt < bal THEN
    INSERT INTO transactions(accountID, amount, transactionType,
       transactionTimestamp)
    VALUES (acctId, amt, 'Withdrawal', NOW());
  END IF;
END $
DELIMITER ;
```

```
SELECT balance FROM accounts WHERE accountID = 1001;
```

*	balance
1	1200.00

Figure 10: Initial Balance 1001

```
SELECT * FROM transactions WHERE accountID = 1001;
```

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10001	1001	200.00	Deposit	2025-09-12 10:05:00
2	10002	1001	100.00	Withdrawal	2025-09-14 11:30:00

Figure 11: Initial Transactions 1001

```
SET @msg = '';
CALL withdrawl(1001, 500.75, @msg);
SELECT @msg;
```

*	@msg
1	Success

Figure 12: Withdrawl message 1001

*	balance
1	699.25

Figure 13: Balance after withdrawl 1001

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10001	1001	200.00	Deposit	2025-09-12 10:05:00
2	10002	1001	100.00	Withdrawal	2025-09-14 11:30:00
3	10022	1001	500.75	Withdrawal	2026-03-05 22:43:08

Figure 14: Transactions After Withdrawal 1001

```
SELECT balance FROM accounts WHERE accountID = 1006;
```

*	balance
1	300.00

Figure 15: Initial Balance 1006

```
SELECT * FROM transactions WHERE accountID = 1006;
```

```
SET @msg = '';
CALL withdrawl(1006, 500.75, @msg);
SELECT @msg;
```

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10010	1006	250.00	Withdrawal	2025-10-11 14:05:00
2	10011	1006	300.00	Deposit	2025-11-20 10:10:00

Figure 16: Initial Transactions 1006

*	@msg
1	Insufficient Funds

Figure 17: Withdrawl message 1006

*	balance
1	300.00

Figure 18: Balance after withdrawl 1006

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10010	1006	250.00	Withdrawal	2025-10-11 14:05:00
2	10011	1006	300.00	Deposit	2025-11-20 10:10:00

Figure 19: Transactions After Withdrawal 1006

6 Transfer Amount

```
DELIMITER $
CREATE PROCEDURE transfer(cName VARCHAR(40), amt DECIMAL(12, 2), OUT
   msg VARCHAR(20))
DETERMINISTIC
BEGIN
  DECLARE custId INT;
  DECLARE sId INT;
  DECLARE cId INT;
  DECLARE bal DECIMAL(12, 2);

  SELECT customerID INTO custId
  FROM customers WHERE fullName = cName;

  SELECT accountID INTO sId
  FROM accounts
  WHERE accountType = 'Savings'
    AND customerID = custId;

```

```
  SELECT accountID INTO cId
  FROM accounts
  WHERE accountType = 'Checking'
    AND customerID = custId;

  SELECT balance INTO bal
  FROM accounts WHERE accountId = cId;

  IF amt <= bal THEN
    BEGIN
    SET msg = 'Success';
    CALL withdrawl(cId, amt, msg);
    CALL deposit(sId, amt);
    END;
  ELSE
    SET msg = 'Insufficient Funds';
  END IF;
END $
DELIMITER ;
```

```
SELECT accountType, balance
FROM accounts
WHERE customerID =
  (SELECT customerID FROM customers WHERE fullName = 'Logan Hall');
```

*	accountType	balance
1	Checking	800.00
2	Savings	1600.00

Figure 20: Logan's Accounts

```
SELECT *
FROM transactions
WHERE accountID IN (
  SELECT accountID FROM accounts
  WHERE customerID = (
    SELECT customerID FROM customers WHERE fullName = 'Logan Hall')
  );
```

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10020	1014	350.00	Deposit	2025-11-11 10:05:00

Figure 21: Logan's Transactions

```sql
SET @msg = '';
CALL transfer('Logan Hall', 450.75, @msg);
SELECT @msg;
```

*	@msg
1	Success

Figure 22: Transfer message

*	accountType	balance
1	Checking	349.25
2	Savings	2050.75

Figure 23: Logan's Accounts After Transfer

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10020	1014	350.00	Deposit	2025-11-11 10:05:00
2	10023	1014	450.75	Withdrawal	2026-03-06 00:13:45
3	10024	1015	450.75	Deposit	2026-03-06 00:13:45

Figure 24: Logan's Transactions After Transfer

```sql
SELECT accountType, balance
FROM accounts
WHERE customerID =
  (SELECT customerID FROM customers WHERE fullName = 'Sophia Turner');
```

*	accountType	balance
1	Checking	300.00
2	Savings	3100.25

Figure 25: Sophia's Accounts

```sql
SELECT *
FROM transactions
WHERE accountID IN (
  SELECT accountID FROM accounts
  WHERE customerID = (
    SELECT customerID FROM customers WHERE fullName = 'Sophia Turner')
  );
```

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10010	1006	250.00	Withdrawal	2025-10-11 14:05:00
2	10011	1006	300.00	Deposit	2025-11-20 10:10:00
3	10008	1005	400.00	Deposit	2025-10-06 13:22:00
4	10009	1005	200.00	Withdrawal	2025-11-21 14:00:00
5	10021	1005	100.25	Deposit	2026-03-05 22:20:27

Figure 26: Sophia's Transactions

```sql
SET @msg = '';
CALL transfer('Sophia Turner', 450.75, @msg);
SELECT @msg;
```

*	@msg
1	Insufficient Funds

Figure 27: Transfer message

*	accountType	balance
1	Checking	300.00
2	Savings	3551.00

Figure 28: Sophia's Accounts After Transfer

*	transactionID	accountID	amount	transactionType	transactionTimestamp
1	10010	1006	250.00	Withdrawal	2025-10-11 14:05:00
2	10011	1006	300.00	Deposit	2025-11-20 10:10:00
3	10008	1005	400.00	Deposit	2025-10-06 13:22:00
4	10009	1005	200.00	Withdrawal	2025-11-21 14:00:00
5	10021	1005	100.25	Deposit	2026-03-05 22:20:27

Figure 29: Sophia's Transactions After Transfer